Testing SEOS with PyTest

Page Content

Goals

- We care about test case results, not about how out our test tools work
- test tools have to follow the need of the test cases, not the other way around
- test case documentation is mainly about the test case and not about the implementation on the test tools

Improvements

- **I.1:** test log parser needs to be smarter
- **I.2:** QEMU/Proxy as test case parameter seem strange
- **I.3:** test failure output does not really helper understanding the actual failure
- **I.4:** too slow
- **I.5:** test logic split in backend (test system, C) and frontend (test script., Python) consequences are:
  - anti-pattern of maintenance of code (2 places)
  - too many test systems to build will overload again the CI server eventually

---

Repository

Both C API and the python generator can be found in the following location:

https://bitbucket.hensoldt-cyber.systems/projects/SSDK/repos/dev-scripts/browse/tests

Proposed Improvements

The current test setup relies on executing a test written in C against a set of functions of a module (could be an API or a component) in QEMU and then parsing the output of such a test-run with a test script in Python (on the host).

The test script as such has only two functions:

1. Find a string in the test output indicating a test has succeeded
2. If a test was not successful, fail with a message indicating what went wrong **in the underlying C test**

To improve on the current state, where an assertion in the pytest script usually does not provide much information about what went wrong in the C test, we propose a few conventions which incidentally also allow to automate parts of the test writing.

Based on these conventions, we can automatically generate the pytest script and use it with an improved test_parser.py module to enhance pytest utility.

Conventions

In order to make mapping C test files, tests within these files and the corresponding functions in the pytest script as easy as possible it makes sense to follow some simple conventions.

The following is an example from the SEOS Crypto API (see current branch SEOS_0_9_crypto_api in seos_tests/) for illustration purposes:

There are several test files for the different sub-modules of API:

```
test_SeosCryptoApi_Agreement.c
test_SeosCryptoApi_Cipher.c
test_SeosCryptoApi.c
etc.
```

Actual tests within test_SeosCryptoApi_Cipher.c are named like this:

```
test_SeosCryptoApi_Agreement_init_pos()
test_SeosCryptoApi_Agreement_init_neg()
test_SeosCryptoApi_Agreement_agree_neg()
etc.
```

We see that all tests start with "test_". Furthermore, they make understanding what the test is supposed to do very easy:

1. **test**_SeosCryptoApi_Cipher_init_**pos**() tests SeosCryptoApi_Cipher_init() with **valid** parameters (positive test).
2. **test**_SeosCryptoApi_Cipher_init_**neg**() tests SeosCryptoApi_Cipher_init() with **invalid** parameters to check if the API handles these cases correctly (negative test).
3. ...

The general structure of any such positive test looks like this:

```
static void test_SeosCryptoApi_Agreement_init_pos()
{
        TEST_START();

        ...
        TEST_SUCCESS(SeosCryptoApi_Agreement_init(&api, &obj, &goodConfig0));
        TEST_SUCCESS(SeosCryptoApi_Agreement_free(&obj));

        TEST_SUCCESS(SeosCryptoApi_Agreement_init(&api, &obj, &goodConfig1));
        TEST_SUCCESS(SeosCryptoApi_Agreement_free(&obj));

        TEST_SUCCESS(SeosCryptoApi_Agreement_init(&api, &obj, &goodConfig2));
        TEST_SUCCESS(SeosCryptoApi_Agreement_free(&obj));
        ...

        TEST_SUCCESS(SeosCryptoApi_Agreement_readParam(&api, &param));
        TEST_TRUE(param == 1234);
        ...

        TEST_FINISH();
}
```

Alternatively, a negative test looks like this:

```
static void test_SeosCryptoApi_Agreement_init_neg()
{
        TEST_START();

        ...
        TEST_INVAL_PARAM(SeosCryptoApi_Agreement_init(&api, &obj, &badConfig0));

        TEST_INVAL_PARAM(SeosCryptoApi_Agreement_init(&api, &obj, &badConfig1));

        TEST_INVAL_PARAM(SeosCryptoApi_Agreement_init(&api, &obj, &badConfig2));
        ...

        TEST_FINISH();
}
```

In such a test function, TEST_XXX() asserts that an expression equals an error code:

- TEST_SUCCESS(ex) asserts ex == SEOS_SUCCESS,
- TEST_INVAL_PARAM(x) asserts ex == SEOS_ERROR_INVALID_PARAM
- ...

There is one TEST_XXX() macro for every generic seos_err_t error code. TEST_TRUE() simply asserts that an expression is true.

In order to empower pytest, when an assertion is hit, TEST_XXX() also outputs the last test function that was called. For this, TEST_START() has to be called at the beginning of each test.

Finally, TEST_FINISH() also prints a string indicating the successful completion of the test function, based on the functions name. An example output of the complete Crypto API test thus looks like this:

```
..
!!! test_SeosCryptoApi_init_neg: OK
!!! test_SeosCryptoApi_free_neg: OK

Testing Crypto API in SeosCryptoApi_Mode_LIBRARY mode:
!!! test_SeosCryptoApi_Agreement_init_pos(api->mode=0,allowExport=1): OK
!!! test_SeosCryptoApi_Agreement_init_neg(api->mode=0,allowExport=1): OK
!!! test_SeosCryptoApi_Agreement_agree_neg(api->mode=0,allowExport=1): OK
...
```

We see that we can also have tests which are parametrized; currently, TEST_START() takes up to two integer arguments, which are then mapped to such a test output.

Generating pytest files

If all tests are structured as proposed above and use the TEST_XXX() macros, we can use a python script to generate a the corresponding pytest script.

For this, we simply pipe the output of QEMU of a successful test run into the script, which then profiles the test run by extracting test result messages *and measuring the run time* between those messages to determine individual timeout values.

The script will capture the output of QEMU but also print it so it is clear where in the test we are:

```
bendri01@b:~/dev/projects/seos_tests.cryptoserver$ src/run_qemu.sh test_crypto_api | ../../dev-scripts/tests
/generate_script.py -o test_crypto_api.py


============================================== CAPTURING OUTPUT ==============================================
         ...
...... |
...... | start_threads@main.c:1798  Starting crypto_crypto_Crypto_0000_tcb...
...... |
...... | start_threads@main.c:1798  Starting crypto_crypto_SeosCryptoRpcServer_0000_tcb...
...... |
...... | start_threads@main.c:1798  Starting test_crypto_test_crypto_0_control_tcb...
...... |
...... | start_threads@main.c:1798  Starting test_crypto_test_crypto_0_fault_handler_tcb...
...... |
...... | main@main.c:2018 We used 1276 CSlots (0.00% of our CNode)
...... |
...... | main@main.c:2019 Done; suspending...
...... |
+01.7s | !!! test_SeosCryptoApi_init_neg: OK
+00.0s | !!! test_SeosCryptoApi_free_neg: OK
...... |
...... | Testing Crypto API in SeosCryptoApi_Mode_LIBRARY mode:
+00.0s | !!! test_SeosCryptoApi_Agreement_init_pos(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_init_neg(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_agree_neg(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_free_pos(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_free_neg(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_agree_buffer(api->mode=0,allowExport=1): OK
+00.0s | !!! test_SeosCryptoApi_Agreement_do_DH(api->mode=0,allowExport=1): OK
+00.4s | !!! test_SeosCryptoApi_Agreement_do_ECDH(api->mode=0,allowExport=1): OK
+00.2s | !!! test_SeosCryptoApi_Agreement_do_DH_rnd(api->mode=0,allowExport=1): OK
         ...
```

In this output we see that every test output gets marked with duration that test required in this particular test run.

The script will then write a file (here test_crypto_api.py) which contains the pytest code and looks like this:

```
import pytest, sys
import test_parser as parser

TEST_NAME = 'test_crypto_api'

# AUTOGENERATED TEST FUNCTIONS BELOW ----------------------------------------

def test_SeosCryptoApi_init_neg(boot):
    """
    Negative tests for SeosCryptoApi_init(), covering the invalid ways of using this
    function thus verifying that it returns error codes instead of crashing
    """
    parser.check_test(boot(TEST_NAME), timeout=12, 'test_SeosCryptoApi_init_neg')

def test_SeosCryptoApi_free_neg(boot):
    """
    Negative tests for SeosCryptoApi_free(), covering the invalid ways of using this
    function thus verifying that it returns error codes instead of crashing
    """
    parser.check_test(boot(TEST_NAME), timeout=1, 'test_SeosCryptoApi_free_neg')

def test_SeosCryptoApi_Agreement_init_pos(boot):
    """
    Positive tests for SeosCryptoApi_Agreement_init(), covering the valid ways of
    using this function.
    """
    parser.check_test(boot(TEST_NAME), timeout=1, 'test_SeosCryptoApi_Agreement_init_pos', 'api->mode=0,
allowExport=1')
    parser.check_test(boot(TEST_NAME), timeout=1, 'test_SeosCryptoApi_Agreement_init_pos', 'api->mode=1,
allowExport=1')
    parser.check_test(boot(TEST_NAME), timeout=1, 'test_SeosCryptoApi_Agreement_init_pos', 'api->mode=2,
allowExport=1')
...
```

We can see that every pytest function name matches the underlying C test function and consists of a one or more calls to check the output of the test run based on the name of the test. The timeout parameter for each call to check_test() is the result of the individual timing measurement multiplied by a factor to accommodate for different behavior on the CI system.

Please also note that test comments can be "guessed" based on the proposed convention.

This pytest file uses the new test_parser.py module, which parses actual test outputs in order to produce pytest failures which are more meaningful than what we currently have, see below.

Discussion

If a test fails in pytest, the resulting pytest output now indicates what went wrong in the underlying C test. Here is an example how this looks like:

```
=================================================================================================== FAILURES
===================================================================================================
_____________________________________________________________________________
test_SeosCryptoApi_Agreement_free_pos[/host]
_____________________________________________________________________________

boot = <function use_qemu_with_proxy.<locals>.<lambda> at 0x7ff380fe19d8>

    def test_SeosCryptoApi_Agreement_free_pos(boot):
        """
        <TODO: Describe here what the test does>
        """
>       parser.check_test(boot(TEST_NAME), 1, 'test_SeosCryptoApi_Agreement_free_pos', 'api->mode=0,
allowExport=1')
E       Failed: SeosCryptoApi_Agreement_free(&obj) == SEOS_ERROR_INVALID_PARAMETER (/host/src/src/tests
/test_crypto_api/components/TEST_CRYPTO/src/test_SeosCryptoApi_Agreement.c:
test_SeosCryptoApi_Agreement_free_pos: 337)

test_crypto_api.py:44: Failed
_____________________________________________________________________________
test_SeosCryptoApi_Agreement_free_neg[/host]
_____________________________________________________________________________

boot = <function use_qemu_with_proxy.<locals>.<lambda> at 0x7ff380fe19d8>

    def test_SeosCryptoApi_Agreement_free_neg(boot):
        """
        <TODO: Describe here what the test does>
        """
>       parser.check_test(boot(TEST_NAME), 1, 'test_SeosCryptoApi_Agreement_free_neg', 'api->mode=0,
allowExport=1')
E       Failed: Aborted because test_SeosCryptoApi_Agreement_free_pos already failed
```

We see here that the underlying assertion in the C test is shown in the pytest failure. Furthermore, pytest does not try and wait for more test results because, in this case, a single assertion failure aborts the whole test run (this is different for multi-threaded tests).

This proposal addresses improvement requests as follows:

- **I.1:** Parsing of output is smarter because it also prevents to wait for test results if there has already been a failure; additionally, timeout values are now more fitting to the individual tests.
- **I.3:** Pytest failures are mapped to failures of the underlying C test, so diagnosis is much easier.
- **I.5:** If some conventions are followed, the pytest file can be generated from a successful run of the C test via the described script.

Scripts and macros

In order to use the proposed conventions and tools, the following are required:

1. TestMacros.h contains the TEST_xxx() macros
2. generate_script.py generates pytest files based on succesful test runs
3. test_parser.py is a module used by the newly generated pytest files to intelligently parse test outputs

All of them can be found in dev-scripts.

There is also a script to mostly automate the re-writing of tests so that they use TEST_XXX(). This script is not in the best shape, but available on request.

Applicability of test scheme to existing scripts

The following table list the components/modules/libraries for which the new test concept could be applied:

Module	Owner	PY file	New scheme applies	Comments
Configuration	Unknown User (matbuh01)	test_config_server_fs_backend.py	✓	Should be applicable

		test_config_server.py	✅	Should be applicable
ChanMux	Unknown User (carpin01)	test_chanmux.py	✅	Should be applicable (Guessed by Unknown User (bendri01))
Crypto	Unknown User (bendri01)	test_crypto_api.py	✅	Already done
CryptoServer	Unknown User (bendri01)	test_crypto_api.py	✅	Already done
Partition Manager	Unknown User (sebeck01)	test_partition_manager.py	✅	Should be applicable
Filesystem	Unknown User (sebeck01)	test_filesystem_as_lib.py	✅	Should be applicable
	Unknown User (sebeck01)	test_seos_filestream.py	✅	Should be applicable (but currently this test is not working)
KeyStore	Axel Heider	test_keystore_demo.py	✅	should work, needs to be assessed practically
		test_keystore_preprovisioned.py	✅	should work, needs to be assessed practically
		test_keystore.py	✅	should work, needs to be assessed practically
Logger	Unknown User (slakwa01)	test_logserver.py	❌	Logger needs a full access to the stdout, so the test framework cannot be used.
Network	Thomas Böhm	test_network_api.py	❌	Test stimuli are applied from the pytest file.
Proxy NVM	Unknown User (sebeck01)	test_proxy_nvm.py	✅	Should be applicable
Tls	Unknown User (bendri01)	test_tls_api.py	✅	Already done
TLSServer	Unknown User (bendri01)	test_tlsserver.py	✅	Already done